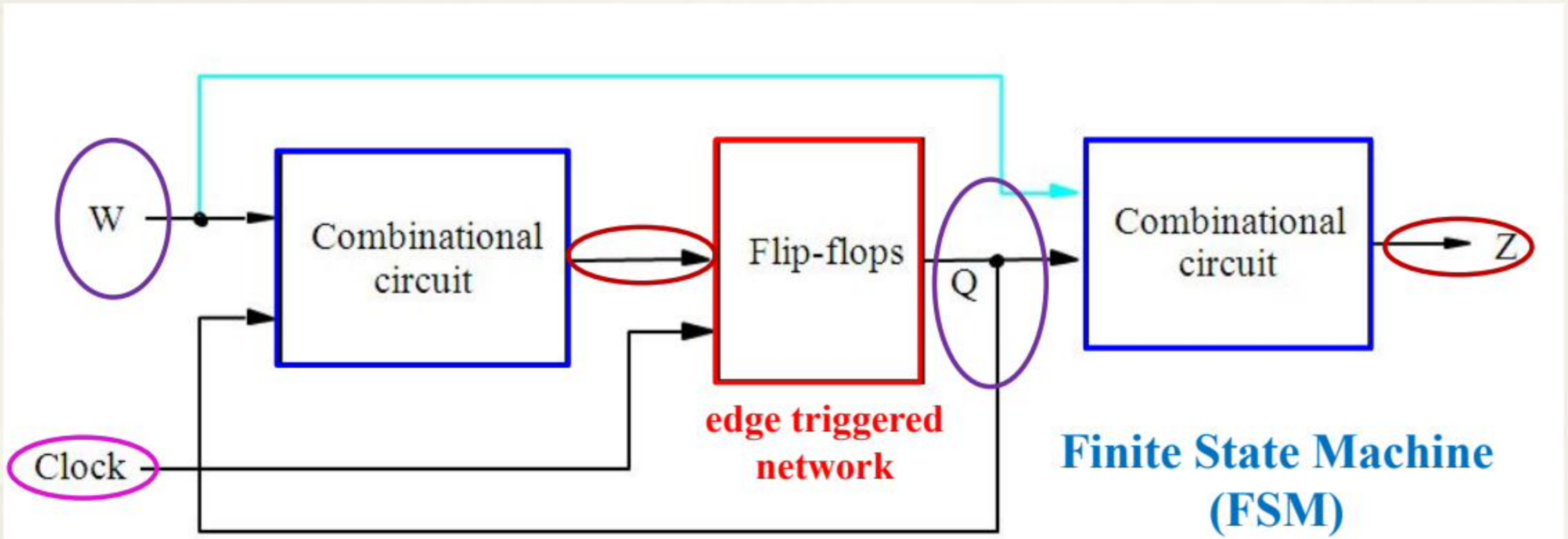
CH6 Synchronous Sequential Logic Circuits

6.1 A general structure



- Outputs may depend on both inputs & present states of flip-flops.
  - External(Primary) outputs(e.g.: Z)
  - Internal outputs: inputs of flip-flops(D, T, JK...).
- Inputs:
  - External(primary) inputs(e.g.: W)
  - Internal inputs: feedback inputs from flip-flops(present state).

Moore type & Mealy type

- Moore type: whose external outputs only depend on the present state.
- Mealy type: whose external outputs depend on both the present state and primary inputs.

6.2 Basic Design Steps

- **Implementation of a "11" sequence detector:**
  - A primary input *w*, and an output, *z*.
  - Serial input and output synchronized by clock signal.
  - A single clk synchronizing rising edge triggered ffs.
  - Z=1 if w=1 during two or more consecutive clock cycles; z=0 otherwise.

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

6.2.1 State diagram

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

- Determine number of states needed.
- Set a starting state in the first place.
- Consider state transitions from the starting state to the last, which covers all the cases.

Assume A is the initial state: w=0 at the beginning; z=0

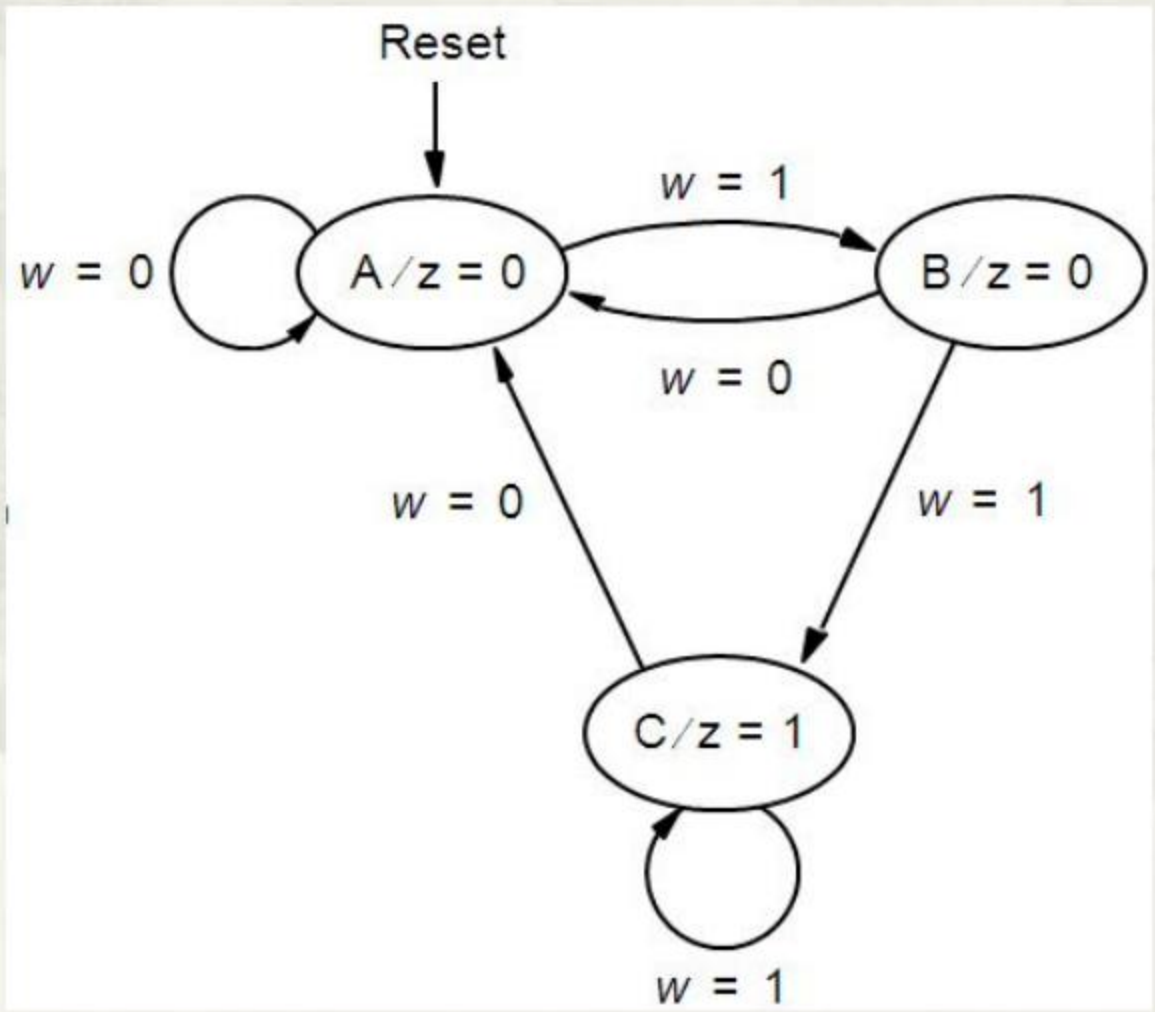
Assume B is the 2nd state: the 1st '1' occurs at the input; z=0

Assume C is the 3rd state: the 2nd '1' occurs at the input; z=1

Z is determined only by the state! **Moore type**

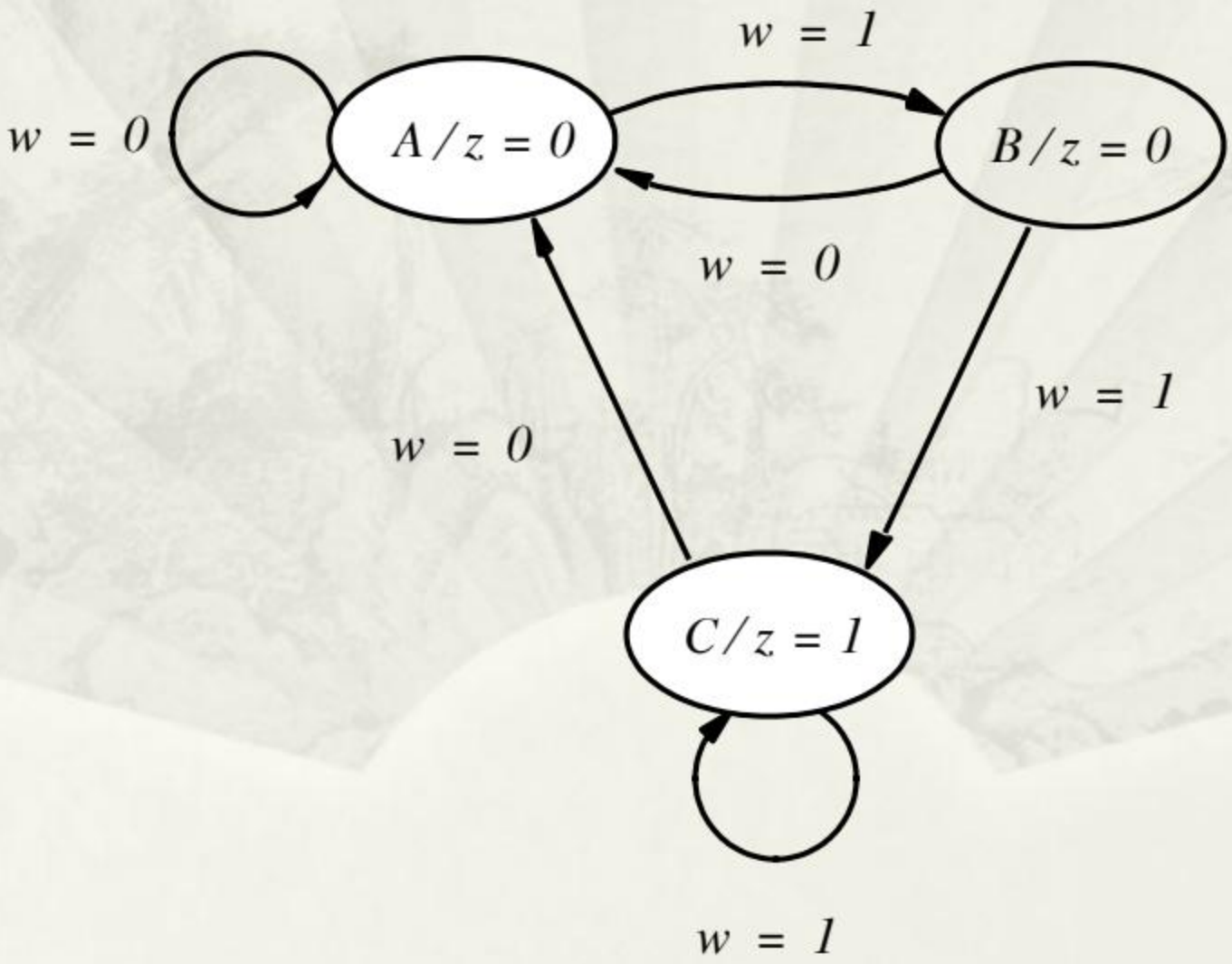
State diagram: the graphical representation

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

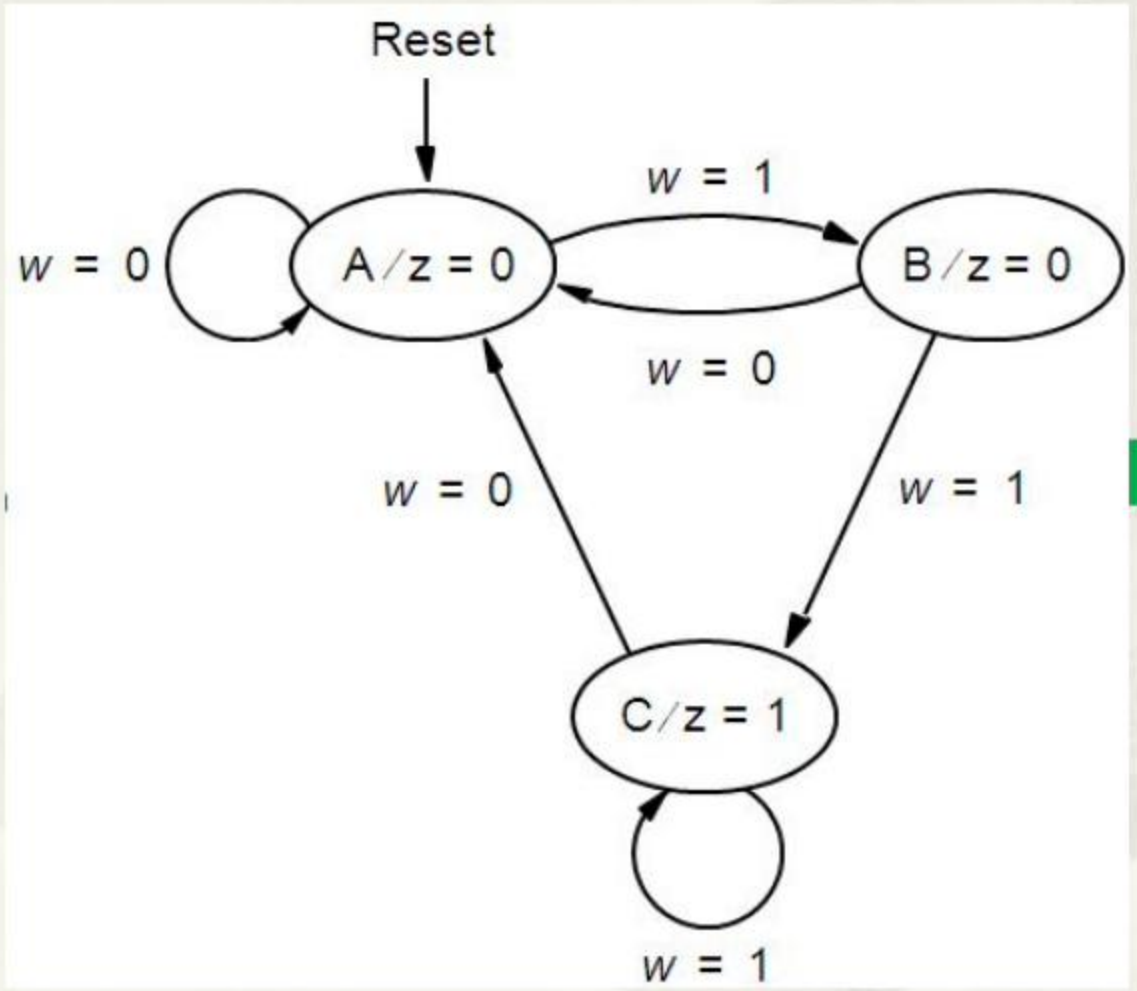


Step-by-step derivation of the state diagram

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0



6.2.2 State table



State diagram

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	C	1

State table

6.2.3 State assignment

- Letter states in the table.
- Binary states in flip-flops.
- Each of letters should be assigned a binary value.
- Consider:

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	C	1

How many flip-flops at least? **2**

2 state variables(assume: y1 & y0) correspond to 2 flip-flops.

00 01 10 11

Choose 3 of them!

More than one assignment!

Choose one of assignments arbitrarily

- Let A=00, B=01, C=10, value of 11 not used.

Present state	Next state		Output
	$w = 0$	$w = 1$	z
A	A	B	0
B	A	C	0
C	A	C	1

	Present state	Next state		Output
		$w = 0$	$w = 1$	
	y_2y_1	Y_2Y_1	Y_2Y_1	z
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

Binary state table

6.2.4 Derivation of Next-state & Output expressions

- A slight modification on the state table.

	Present state	Next state $w = 0$	Next state $w = 1$	Output z
	y_2y_1	Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	*dd*	*dd*	*d*

Before modification

	Present state	Next state $w = 0$	Next state $w = 1$	Output z
	y_2y_1	Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	10	0
	11	*dd*	*dd*	*d*
C	10	*00*	*10*	*1*

After modification

What's the difference?

A combination of K-maps

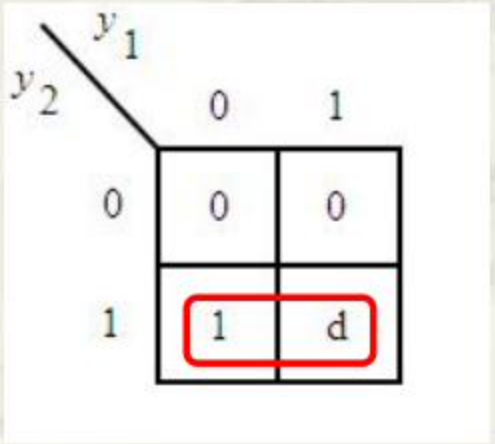
	Present state y_2y_1	Next state $w=0$ Y_2Y_1	Next state $w=1$ Y_2Y_1	Output z
A	00	00	01	0
B	01	00	10	0
	11	*dd*	*dd*	*d*
C	10	*00*	*10*	*1*

Present state y_2y_1	Next state $w=0$ Y_2	Next state $w=1$ Y_2
00	0	0
01	0	1
11	*d*	*d*
10	*0*	*1*

$$Y_2 = wy_1 + wy_2$$

Present state y_2y_1	Next state $w=0$ Y_1	Next state $w=1$ Y_1
00	0	1
01	0	0
11	*d*	*d*
10	*0*	*0*

$$Y_1 = w\overline{y_1}\,\overline{y_2}$$



$$z = y_2$$

Choice of flip-flops

- D? T? JK?
- E.g.: choose D flip-flops.
- Consider D's output expression: $Q^{n+1} = D$

$$Y_2 = wy_1 + w\overline{y_2} \qquad Y_1 = w\overline{y_1}\,\overline{y_2}$$

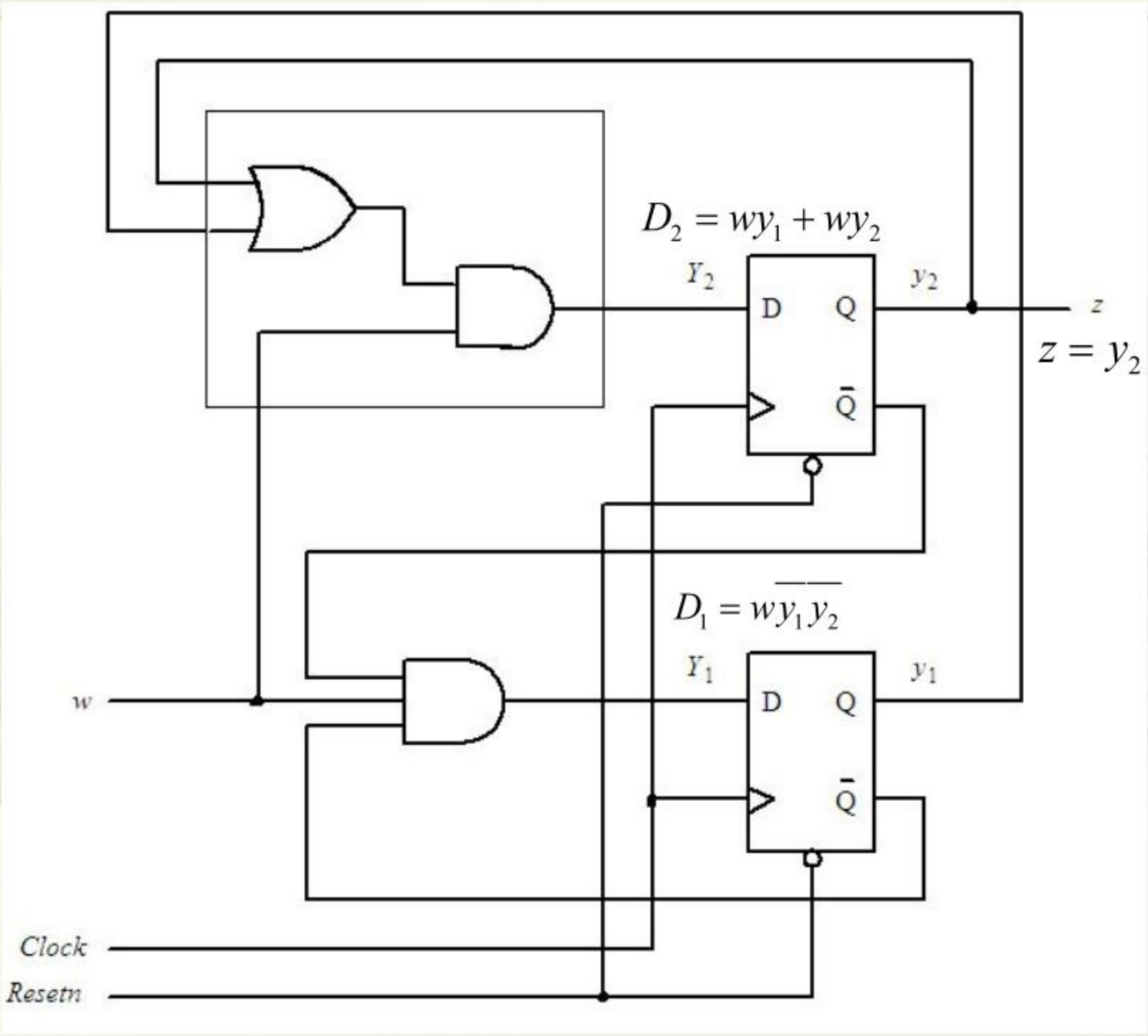
$$\downarrow \qquad\qquad \downarrow$$

$$D_2 = Y_2 = wy_1 + w\overline{y_2} \qquad D_1 = Y_1 = w\overline{y_1}\,\overline{y_2}$$

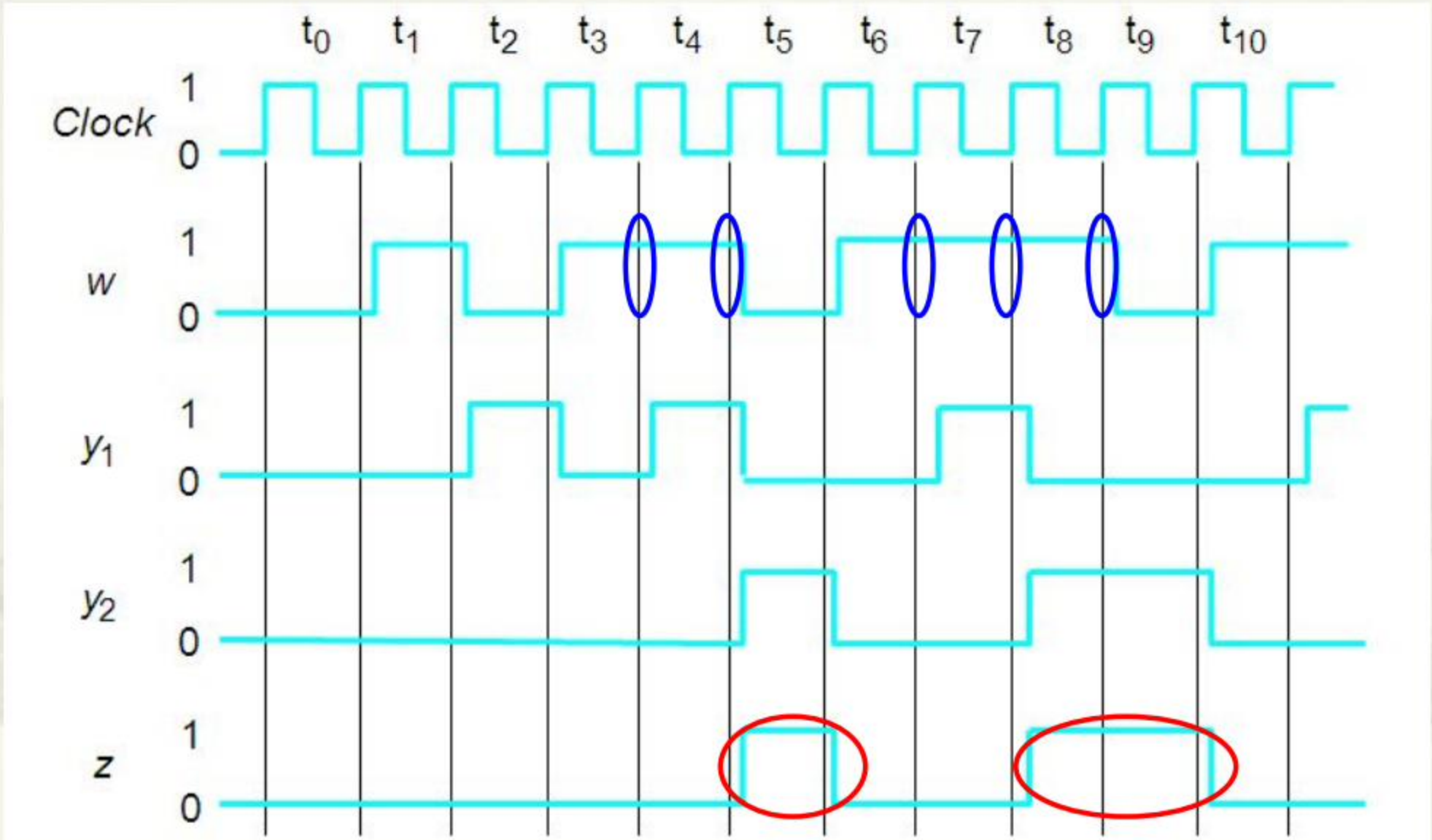
$$\downarrow \qquad\qquad \downarrow$$

Internal outputs(excitation functions)

6.2.5 Build the circuit diagram



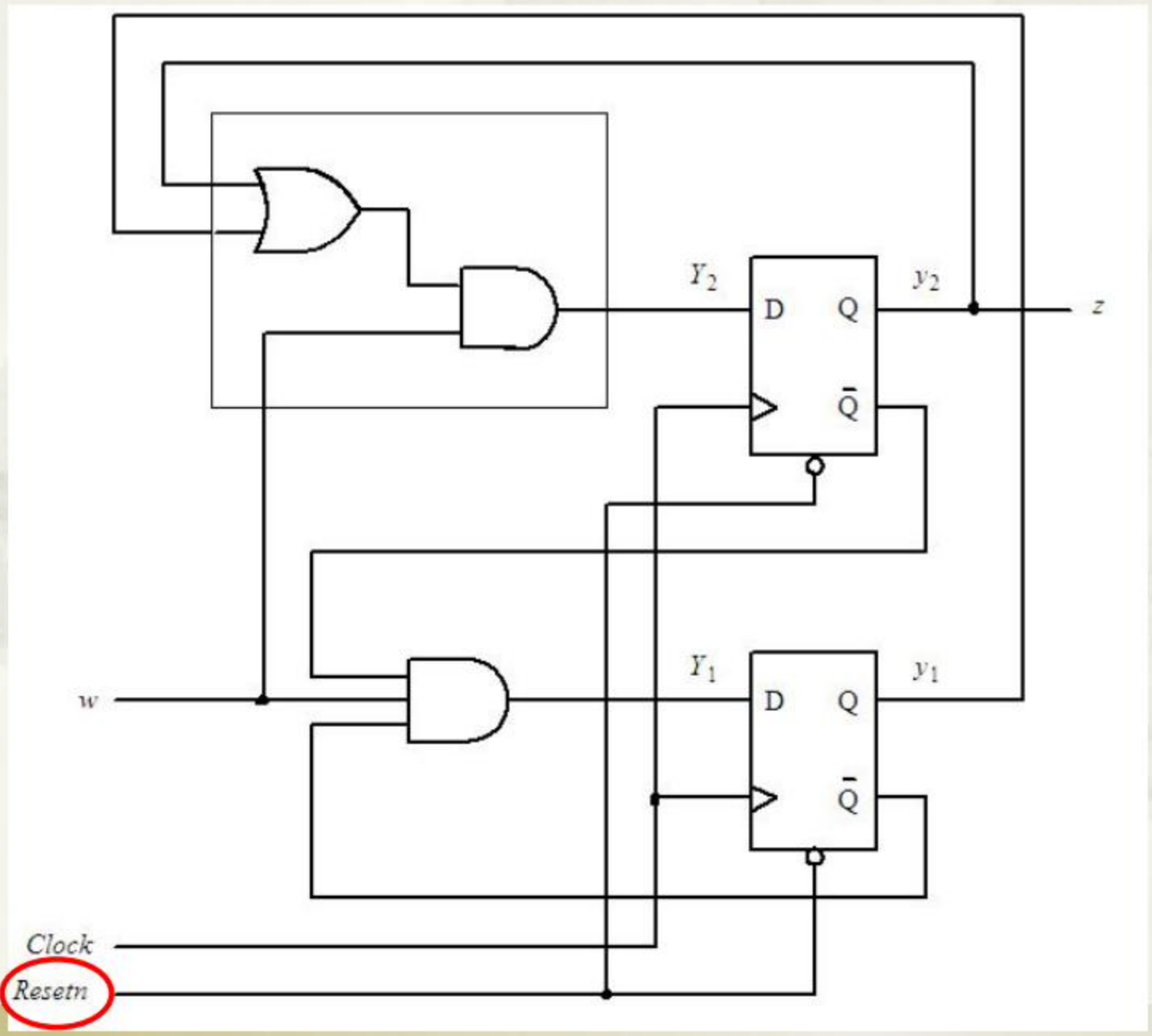
6.2.6 The timing diagram

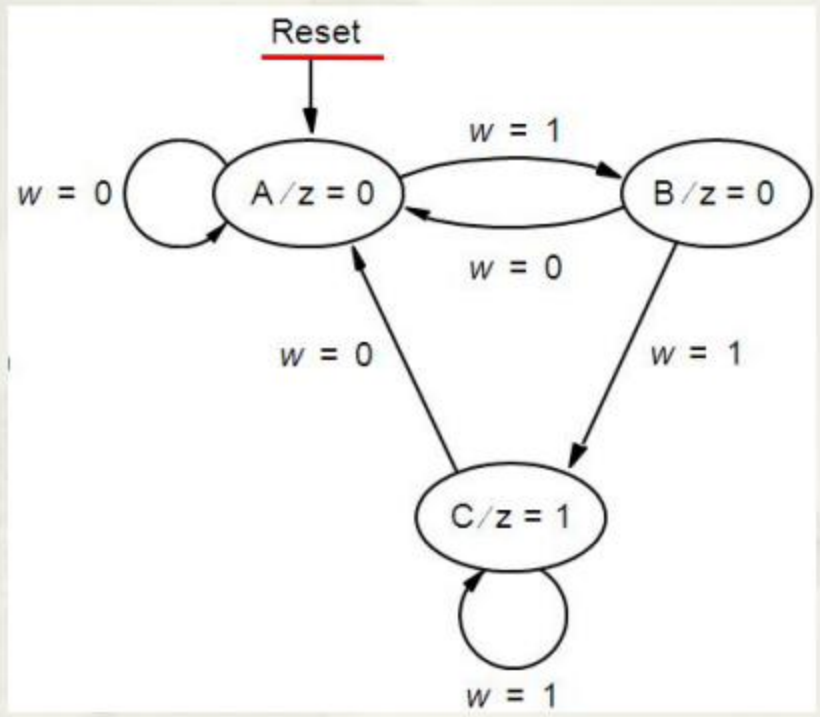


6.2.7 Some detailed issues

1. Set the starting state

- The circuit can start with any state: 00, 01, 10 or *11*
- Setting “00” as the initial state is a good way.





2. The potential impact of unused states

- The "11" state is not used during the derivation of state diagrams.
- However, it may occur typically during the startup.
- It's necessary to check the behavior of the circuit once it's in the "11" state.

$$Y_2 = wy_1 + wy_2 \qquad Y_1 = w\overline{y_1}\,\overline{y_2} \qquad z = y_2$$

Substituting $y_2 = 1$ *and* $y_1 = 1$ *to equations above gives :*

$$Y_2 = w \qquad Y_1 = 0 \qquad z = 1$$

Which indicates :

The circuit changes its state to "00" *if* $w = 0$

The circuit changes its state to "10" *if* $w = 1$

→ **Self-restoring!**

The capability of automatically entering any valid state.

3. State-assignment problem

- A state table corresponds to more than one assignment.
- Consider another one:
  - State A, B, C are represented with valuations y2y1=00, 01, and 11 respectively

	Present state y_2y_1	Next state $w=0$ Y_2Y_1	Next state $w=1$ Y_2Y_1	Output z
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	*dd*	*dd*	*d*

Output z
0
0
1
d

$$Y_1 = D_1 = w$$

$$Y_2 = D_2 = wy_1$$

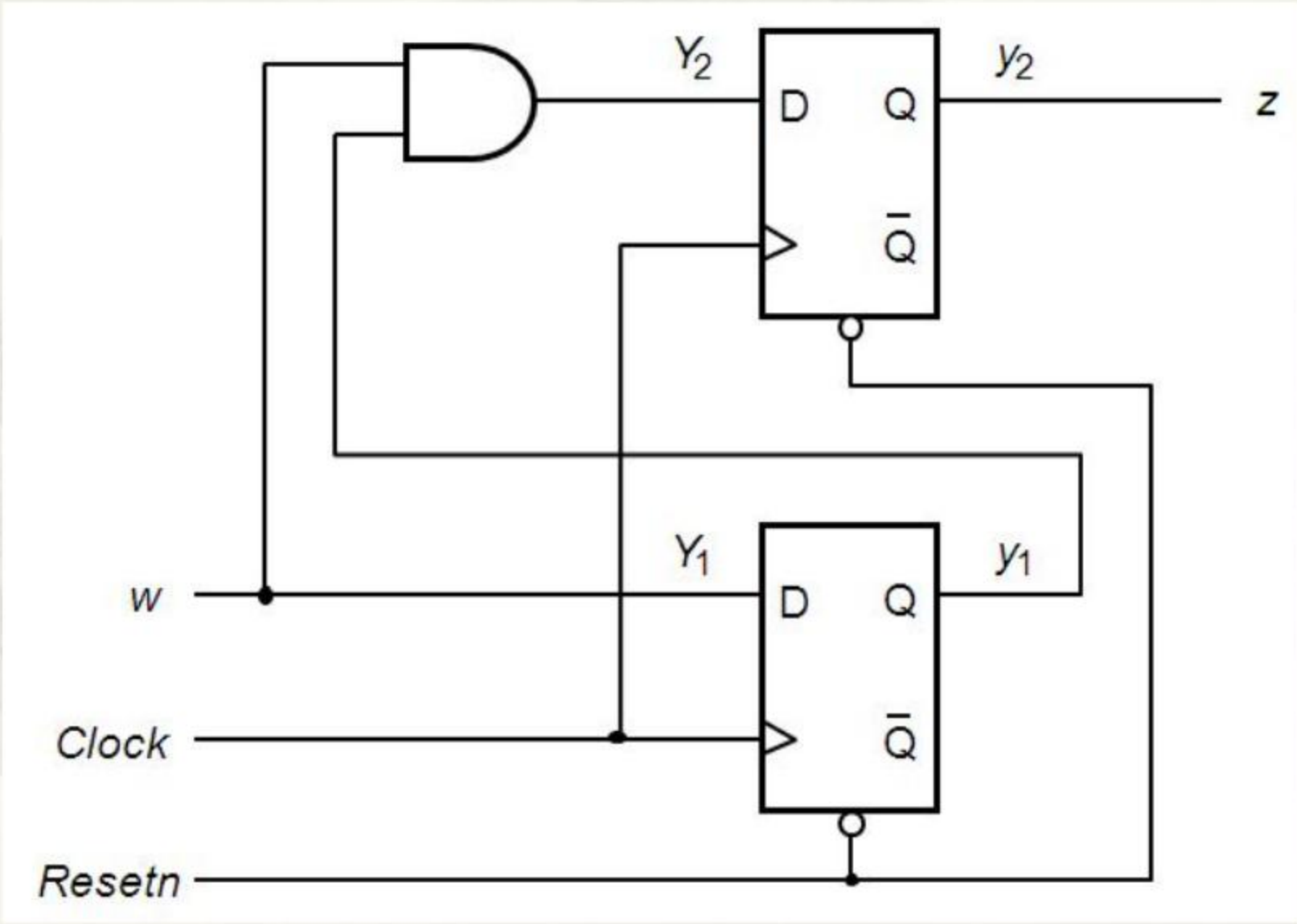
$$z = y_2$$

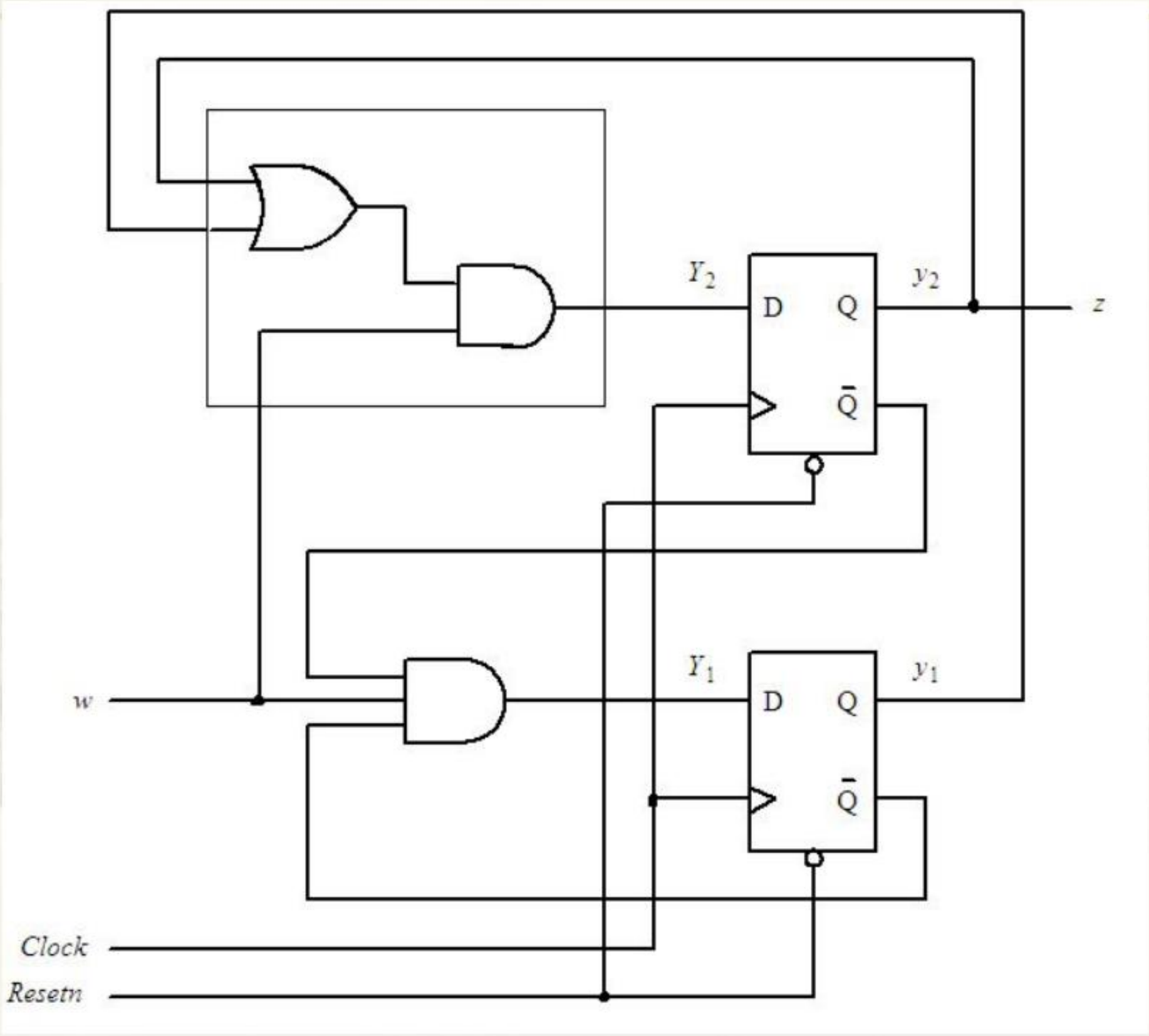
$$Y_2 = wy_1 + wy_2$$

$$Y_1 = w\overline{y_1}\,\overline{y_2}$$

$$z = y_2$$

The corresponding circuit




Y_2
D
Q
y_2
z
$\bar{Q}$
w
Y_1
D
Q
y_1
$\bar{Q}$
Clock
Resetn

4. Selecting other flip-flops

- How about JK flip-flop?

6.2.8 Summary of Design Steps

- Derive the state diagram by setting an initial state.
- State diagram □ state table.
- *State minimization□ optimal state table.*
- State assignment.
- Deriving next-state functions and external output functions
- Choice of flip-flops.
- Deriving excitation functions(D,J,K,T…).
- Build the circuit diagram.
- Analysis of impact of unused states.

6.3 Mealy Type Implementation

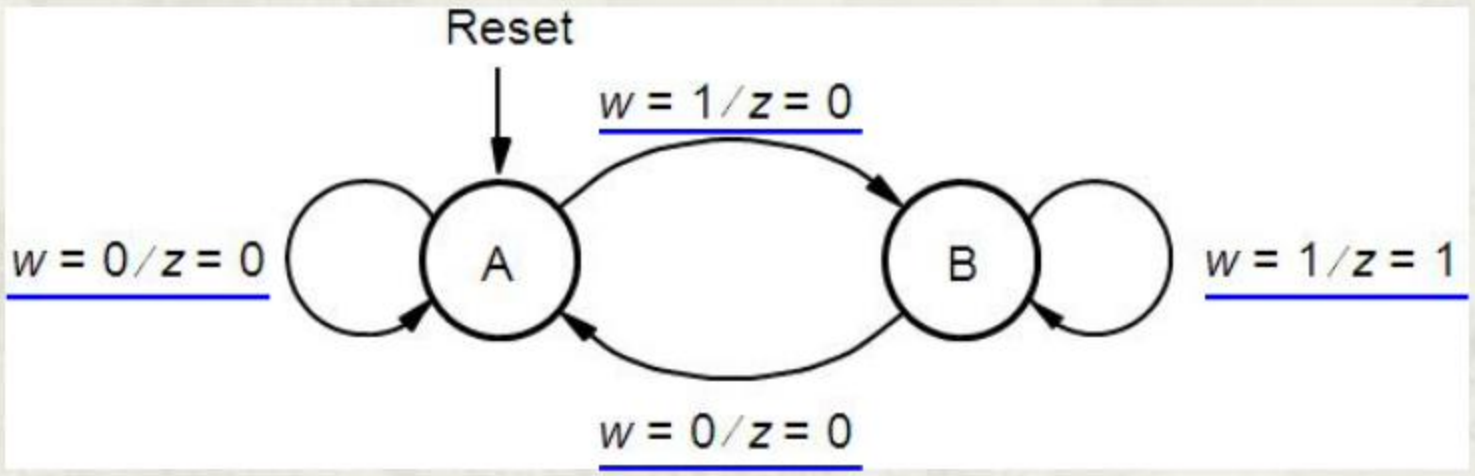
- Implementation of a “11” sequence detector:
  - A primary input *w*, and an output, *z*.
  - Rising edge triggered.
  - Z=1 if w=1 during two or more consecutive clock cycles; z=0 otherwise.

Clockcycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	0	1	0	0	1	1	0

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	1	0	0	1	1	0	0

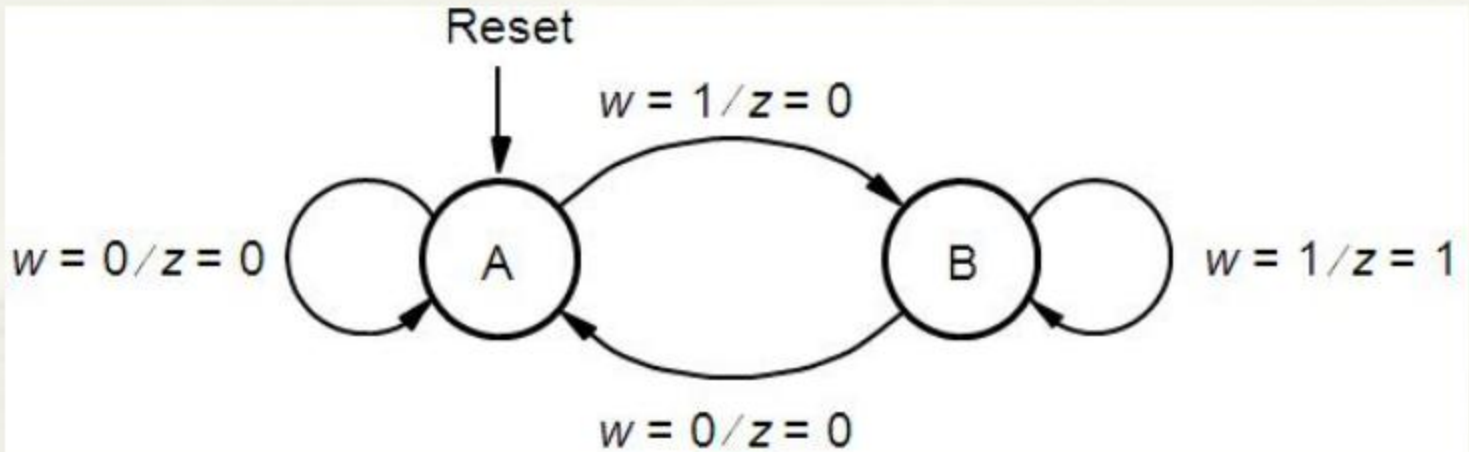
6.3.1 State diagram

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w:	0	1	0	1	1	0	1	1	1	0	1
z:	0	0	0	0	1	0	0	1	1	0	0



- Z is determined by both the inputs and the present state.

6.3.2 State table



Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

6.3.3 State assignment

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

Only 1 flip-flop is needed.

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

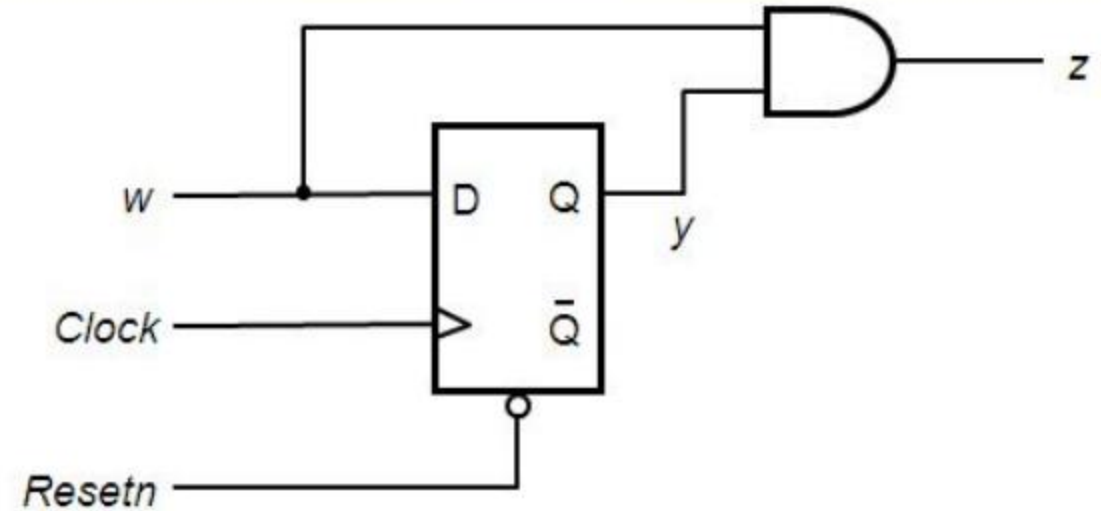
6.3.4 Derivation of Next-state & Output expressions

Present state	Next state		Output	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

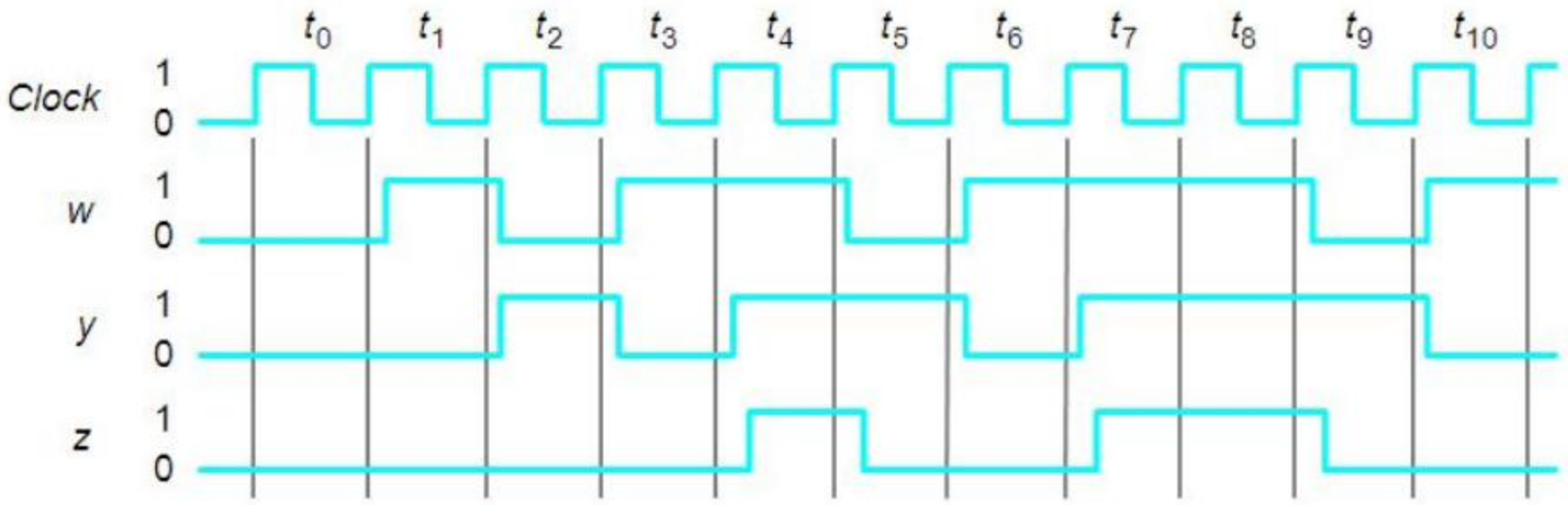
$$Y = D = w$$

$$z = wy$$

6.3.5 Circuit & timing diagram



(a) Circuit



(b) Timing diagram

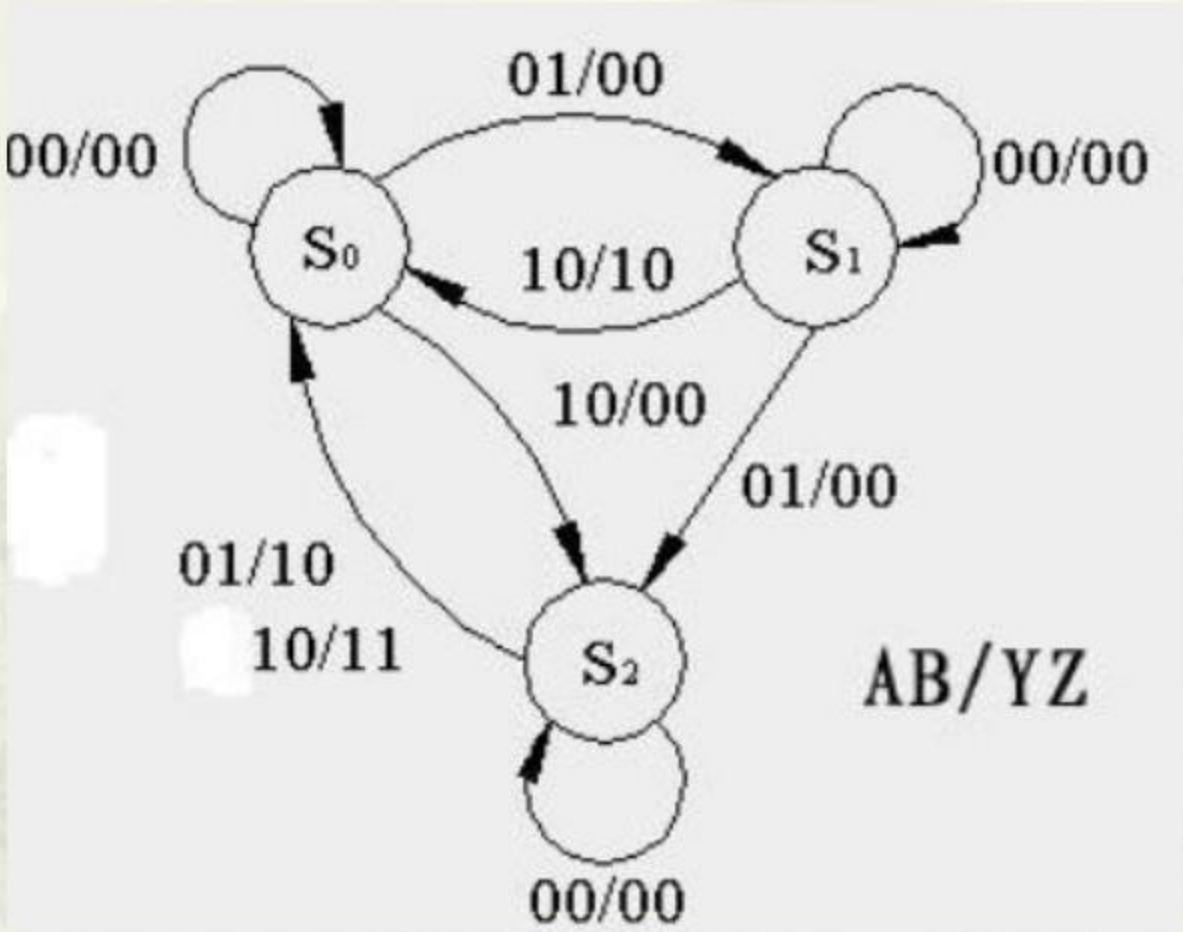
6.5 Another example(Meally Type)

- Implementation of a vending coffee machine:
  - ￥1.5 for each cup.
  - Either a 50 cent coin or a ￥1 coin is accepted.
  - Sell a coffee If ￥1.5 deposited, or return ￥0.5 change besides sending out a coffee if ￥2 deposited.
- Considerations:
  - Which inputs and what do they represent?
  - Which outputs and what do they represent?
  - State changes?

6.5.1 State diagram

- Determine number of primary inputs and external outputs.
  - Set an input A: deposit a ￥1 coin or not.
  - Set an input B: deposit a ￥0.5 coin or not.
  - Set an output Y: sell a cup of coffee or not.
  - Set an output Z: return a ￥0.5 change or not.
- Set a starting state S0
- Set the 2nd state S1: a ￥0.5 coin is deposited.
- Set the 3rd state S2: a ￥1 coin is deposited.
- S0 is the final state as well!

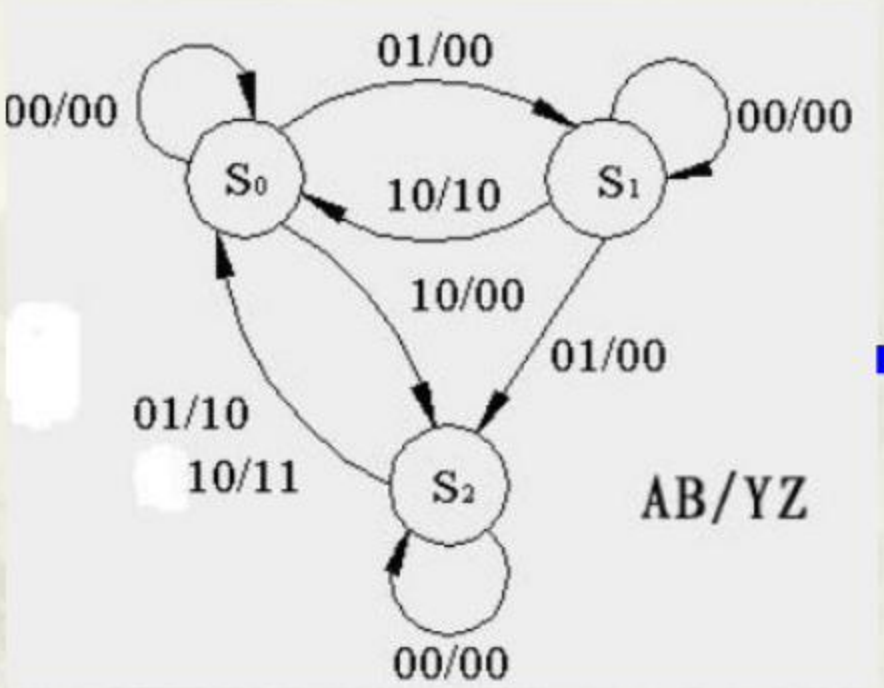
6.5.1 State diagram



AB=11 never occurs

Mealy state diagram

6.5.2 State table



Q	AB			
	00	01	11	10
S0	S0/00	S1/00	d/dd	S2/00
S1	S1/00	S2/00	d/dd	S0/10
S2	S2/00	S0/10	d/dd	S0/11

6.5.3 State assignment

$$Let\ S0 = 00,\ S1 = 01,\ S2 = 10, 11\ left\ unused.$$

Q_1Q_0 \ AB	00	01	11	10
00	00/00	01/00	dd/dd	10/00
01	01/00	10/00	dd/dd	00/10
11	dd/dd	dd/dd	dd/dd	dd/dd
10	10/00	00/10	dd/dd	00/11

Q1&Q0 as state variables

6.5.4 Finding expressions for outputs

Q_1Q_0 \ AB	00	01	11	10
00	00/00	01/00	dd/dd	10/00
01	01/00	10/00	dd/dd	00/10
11	dd/dd	dd/dd	dd/dd	dd/dd
10	10/00	00/10	dd/dd	00/11

$$D_1=Q_1\bar{A}\bar{B}+\bar{Q}_1\bar{Q}_0A+Q_0B$$

$$D_0=\bar{Q}_1\bar{Q}_0B+Q_0\bar{A}\bar{B}$$

$$Y=Q_1B+Q_1A+Q_0A$$

$$Z=Q_1A$$

Implication of using Dffs

Q_1Q_0 \ AB	00	01	11	10
00	0	0	d	1
01	0	1	d	0
11	d	d	d	d
10	1	0	d	0

$D_1=Q_1\bar{A}\bar{B}+\bar{Q}_1\bar{Q}_0A+Q_0B$

Q_1Q_0 \ AB	00	01	11	10
00	0	1	d	0
01	1	0	d	0
11	d	d	d	d
10	0	0	d	0

$D_0=\bar{Q}_1\bar{Q}_0B+Q_0\bar{A}\bar{B}$

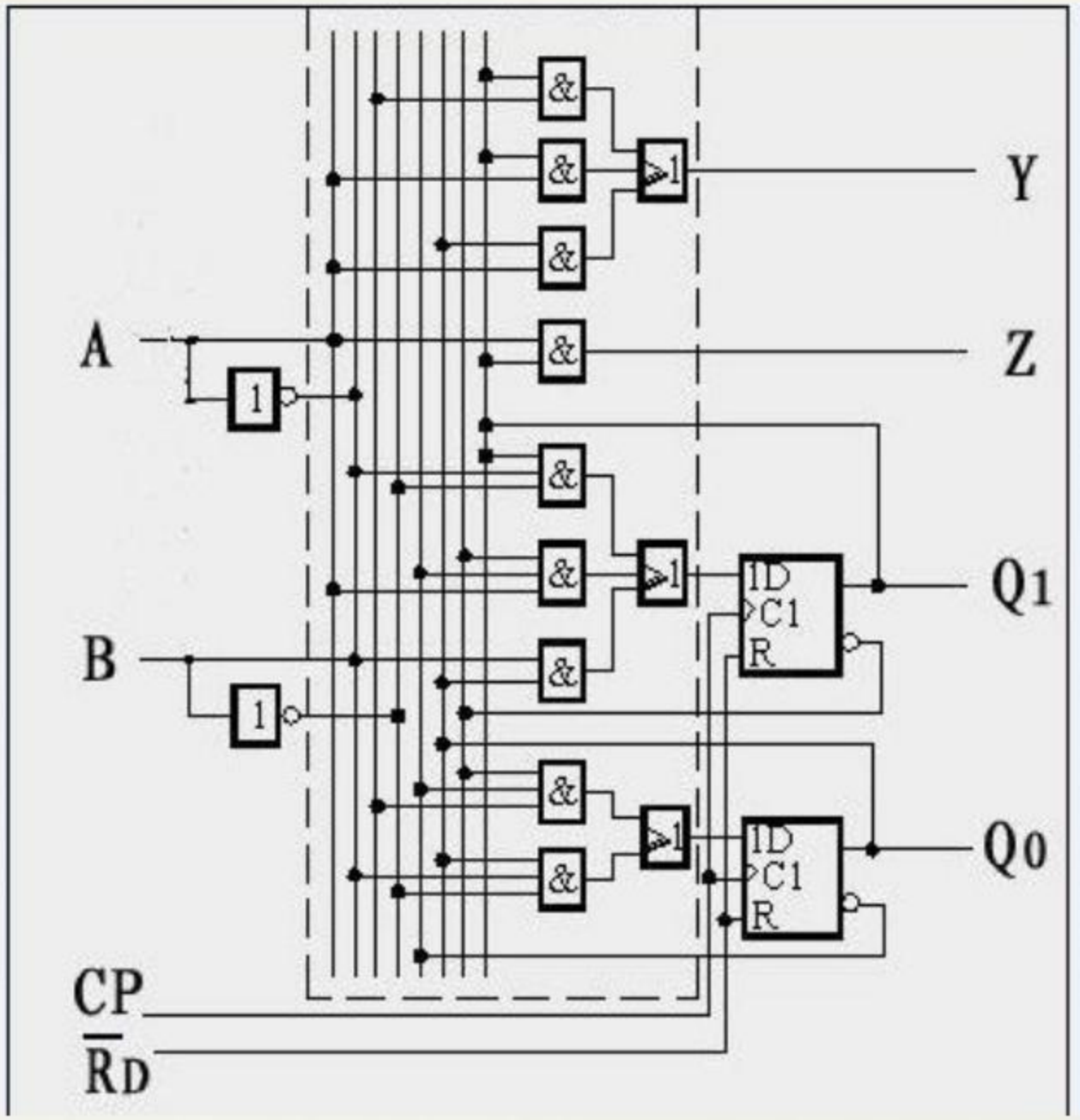
Q_1Q_0 \ AB	00	01	11	10
00	0	0	d	0
01	0	0	d	1
11	d	d	d	d
10	0	1	d	1

$Y=Q_1B+Q_1A+Q_0A$

Q_1Q_0 \ AB	00	01	11	10
00	0	0	d	0
01	0	0	d	0
11	d	d	d	d
10	0	0	d	1

$Z=Q_1A$

6.5.5 The circuit



6.5.6 The impact of the state "11"

- Substitute Q1Q0=11 into

$$D1=Q_1\overline{A}\,\overline{B}+\overline{Q}_1\overline{Q}_0A+Q_0B$$

$$D0=\overline{Q}_1\overline{Q}_0B+Q_0\overline{A}\,\overline{B}$$

$$Y=Q_1B+Q_1A+Q_0A$$

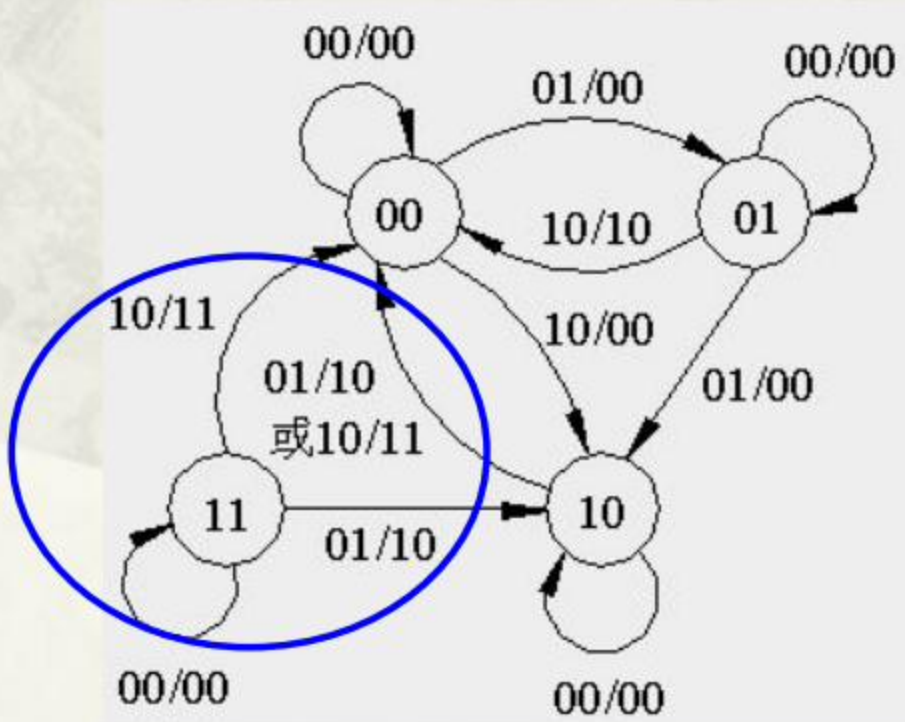
$$Z=Q_1\overline{A}$$

$$D1 = Q1^{n+1} = \overline{A} + B$$

$$D0 = Q0^{n+1} = \overline{A}\,\overline{B}$$

$$Y = A + B$$

$$Z = A$$



Anything wrong?

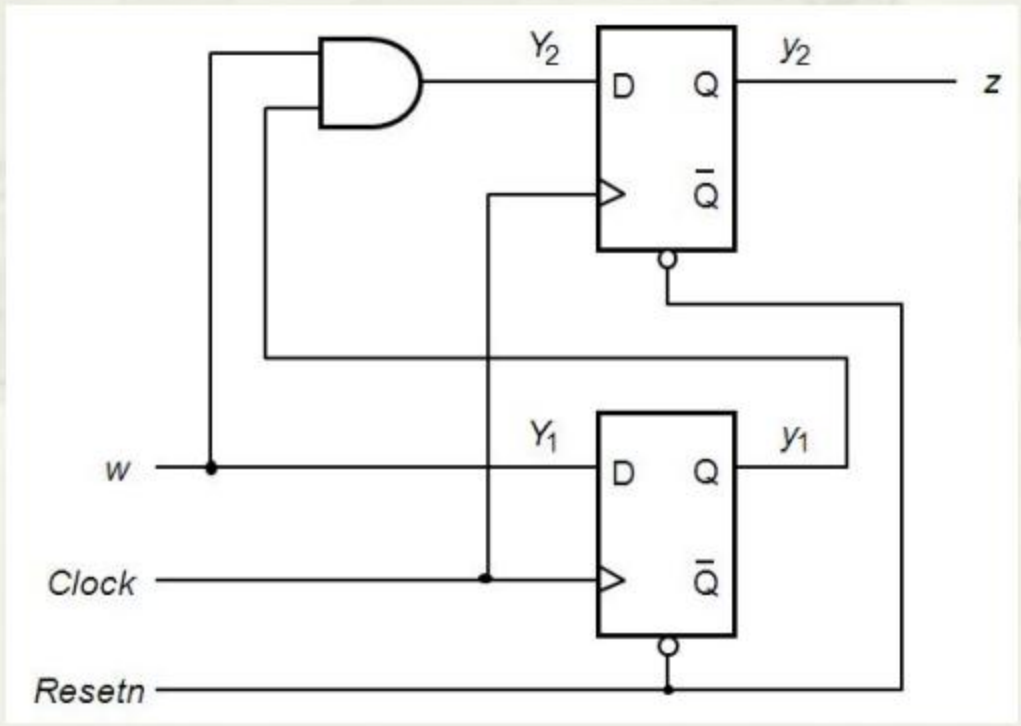
1. Incapable of self-restoring.

2. Errors in charging.

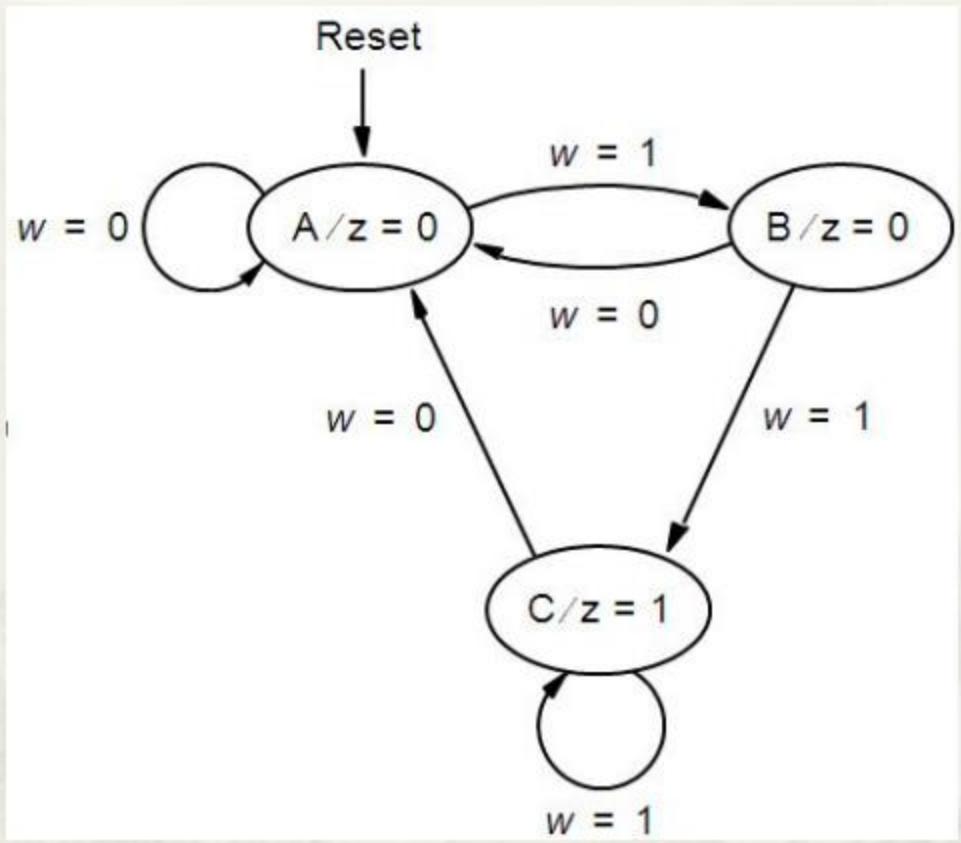
Resetting when power-on is required.

6.4 Implementation in VHDL

```
--Code for Moore type implementation
1   LIBRARY ieee ;
2   USE ieee.std_logic_1164.all ;
3   ENTITY simple IS
4     PORT (   Clock, Resetn, w    : IN STD_LOGIC ;
                              z                    : OUT     STD_LOGIC ) ;
5  END simple ;

7   ARCHITECTURE Behavior OF simple IS
8     TYPE State_type IS (A, B, C) ;
9     SIGNAL y : State_type ;
10  BEGIN
```



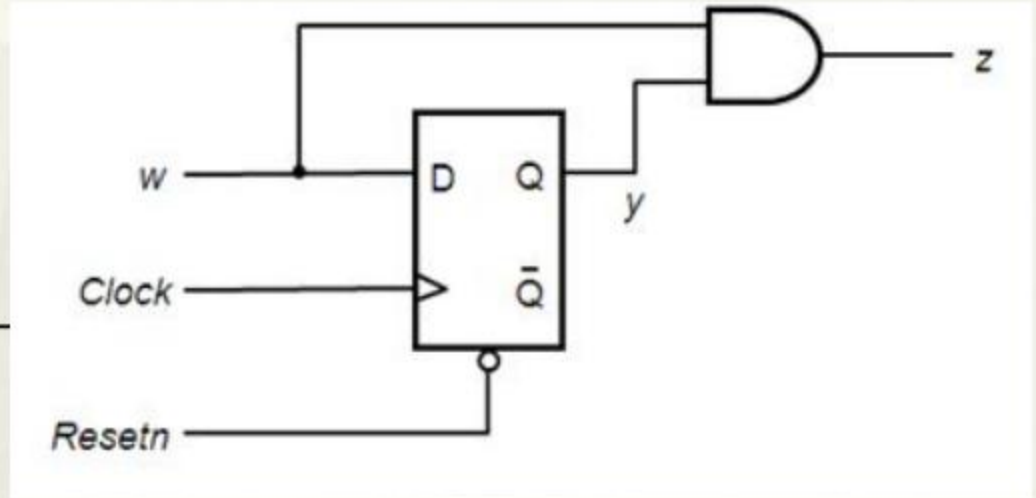
```
13      IF Resetn = '0' THEN
14          y <= A ;
15      ELSIF (Clock'EVENT AND Clock = '1') THEN
16          CASE y IS
17              WHEN A =>
18                  IF w = '0' THEN
19                      y <= A ;
20                  ELSE
21                      y <= B ;
22                  END IF ;
23              WHEN B =>
24                  IF w = '0' THEN
25                      y <= A ;
26                  ELSE
27                      y <= C ;
28                  END IF ;
29              WHEN C =>
30                  IF w = '0' THEN
31                      y <= A ;
32                  ELSE
33                      y <= C ;
34                  END IF ;
35          END CASE ;
36      END IF ;
37  END PROCESS ;
38  z <= '1' WHEN y = C ELSE '0' ;
39 END Behavior ;
```

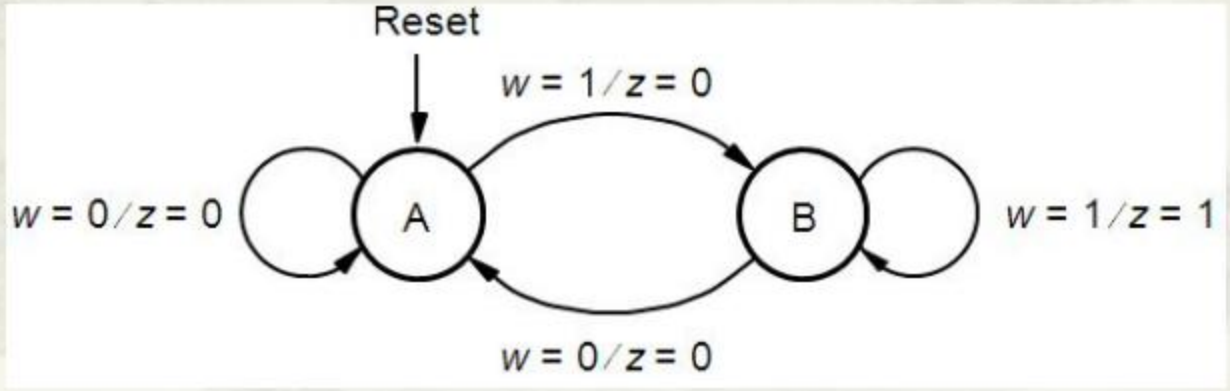


--WHEN OTHERS not required

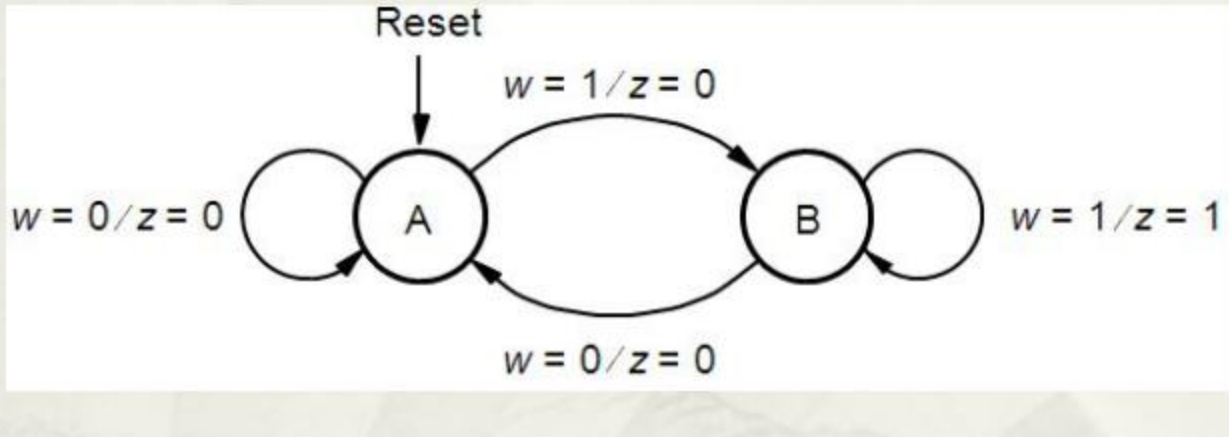
--Code for Mealy type implementation

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY mealy IS
    PORT ( Clock, Resetn, w    : IN  STD_LOGIC ;
           z                   : OUT     STD_
END mealy ;

ARCHITECTURE Behavior OF mealy IS
    TYPE State_type IS (A, B) ;
    SIGNAL y : State_type ;
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
     IF Resetn = '0' THEN
                y <= A ;
```





```
ELSIF (Clock'EVENT AND Clock = '1') THEN
        CASE y IS
            WHEN A =>
                IF w = '0' THEN
                y <= A ;
                ELSE
                y <= B ;
                END IF ;
            WHEN B =>
                IF w = '0' THEN y <= A ;
                ELSE y <= B ;
                END IF ;
        END CASE ;
 END IF ;
END PROCESS ;
```



```
PROCESS ( y, w )
BEGIN
  CASE y IS
        WHEN A =>
              z <= '0' ;
        WHEN B =>
              z <= w ;
    END CASE ;
  END PROCESS ;
END Behavior ;
```